TDD Test Report

Smart Cold Box Sprint 2 • Story 004-1 Accumulate heat exposure • Session 1: heat_increment_s()

TEST CASE SUMMARY

Feature	Expected Output	Test Case	Test Case Name	Dev	Date
004-1 Accumulate heat exposure	0 s (no earlier reading yet)	**TC-01**	First reading adds nothing	Kristranan P. (Areef)	6 Nov 2026
	300 s (9.0 °C at 300 s, next reading at 600 s)	**TC-02**	Earlier reading above the limit counts the whole gap	Kristranan P. (Areef)	6 Nov 2026
	0 s at 8.0 °C, 300 s at 8.1 °C	**TC-03**	Exactly at the limit is not heat	Kristranan P. (Areef)	6 Nov 2026
	0 s (the new read failed: ok = false, temp_c = null)	**TC-04**	Failed read adds nothing	Kristranan P. (Areef)	6 Nov 2026
	0 s, no error (the earlier read failed)	**TC-05**	Earlier reading must be a good reading	Kristranan P. (Areef)	6 Nov 2026
	0 s (clock went from 5000 s back to 10 s, as after power cut)	**TC-06**	Time going backwards adds nothing	Kristranan P. (Areef)	6 Nov 2026

Full test file after the last refactor (all 17 tests in heat_exposure/test_heat_exposure.py): Ran 17 tests in 0.001s OK

Test Case Number: TC-01 | Name: First reading adds nothing (test_first_reading_adds_nothing) Expected

output: 0 s (no earlier reading yet)

RED PHASE

Wrote the test before any code. heat_increment_s() did not exist yet, so the test module could not even import it.

```
def test_first_reading_adds_nothing(self):    self.assertEqual(heat_increment_s(None, reading(0, 9.0), LIMIT), 0)
```

Result: fails (ImportError)

```
$ python -m unittest test_heat_exposure
ERROR: test_heat_exposure (unittest.loader._FailedTest.test_heat_exposure)
ImportError: Failed to import test module: test_heat_exposure
ImportError: cannot import name 'heat_increment_s' from 'heat_exposure'
Ran 1 test in 0.000s
FAILED (errors=1)
```

GREEN PHASE

Smallest code that passes: the function returns 0.

Code in heat_exposure.py:

```
def heat_increment_s(last_good, record, upper_limit_c):    return 0
```

Result: passes

```
$ python -m unittest test_heat_exposure test_first_reading_adds_nothing ... ok
Ran 1 test in 0.000s
OK
```

YELLOW PHASE (REFACTOR)

Added a module header and a docstring that states the contract (always a whole number of seconds, 0 or more). No logic change; the test still passes.

Code after refactoring:

```
def heat_increment_s(last_good, record, upper_limit_c):
    """Seconds of heat to add when record arrives. Always an int >= 0."""    return 0
```

Result: still passes

```
$ python -m unittest test_heat_exposure test_first_reading_adds_nothing ... ok
Ran 1 test in 0.000s
OK
```

> **Test Case Number: TC-02 | Name: Earlier reading above the limit counts the whole gap (test_earlier_reading_above_limit_counts_the_whole_gap)**

Expected output: 300 s (9.0 °C at 300 s, next reading at 600 s)

RED PHASE

The function always returns 0, so the 300 seconds between the two readings were not counted.

```
def test_earlier_reading_above_limit_counts_the_whole_gap(self):    self.assertEqual(heat_increment_s(reading(300, 9.0), reading(600, 5.0), LIMIT), 300)
```

Result: fails

```
$ python -m unittest test_heat_exposure
FAIL: test_earlier_reading_above_limit_counts_the_whole_gap
AssertionError: 0 != 300
Ran 2 tests in 0.001s
FAILED (failures=1)
```

GREEN PHASE

When there is an earlier reading, return the time between the two timestamps.

Code in heat_exposure.py:

```
def heat_increment_s(last_good, record, upper_limit_c):
  """Seconds of heat to add when record arrives. Always an int >= 0."""  if last_good is None:
    return 0  return record["timestamp_s"] - last_good["timestamp_s"]
```

Result: passes

```
$ python -m unittest test_heat_exposure
test_earlier_reading_above_limit_counts_the_whole_gap ... ok
test_first_reading_adds_nothing ... ok
Ran 2 tests in 0.000s
OK
```

YELLOW PHASE (REFACTOR)

Named the time between readings gap_s, so the next rule (look at the earlier temperature) reads clearly. Both tests still pass.

Code after refactoring:

```
def heat_increment_s(last_good, record, upper_limit_c):
  """Seconds of heat to add when record arrives. Always an int >= 0."""  if last_good is None:
    return 0  gap_s = record["timestamp_s"] - last_good["timestamp_s"]  return
gap_s
```

Result: still passes

```
$ python -m unittest test_heat_exposure
test_earlier_reading_above_limit_counts_the_whole_gap ... ok
test_first_reading_adds_nothing ... ok
Ran 2 tests in 0.000s
OK
```

Test Case Number: TC-03 | Name: Exactly at the limit is not heat (test_exactly_at_limit_is_not_heat)

Expected output: 0 s at 8.0 °C, 300 s at 8.1 °C

RED PHASE

The gap was counted no matter what the temperature was, so a reading of exactly 8.0 °C still added 300 s.

```
def test_exactly_at_limit_is_not_heat(self):
  self.assertEqual(heat_increment_s(reading(0, 8.0), reading(300, 8.0), LIMIT), 0)
self.assertEqual(heat_increment_s(reading(0, 8.1), reading(300, 8.0), LIMIT), 300)
```

Result: fails

```
$ python -m unittest test_heat_exposure
FAIL: test_exactly_at_limit_is_not_heat
AssertionError: 300 != 0
Ran 3 tests in 0.001s
FAILED (failures=1)
```

GREEN PHASE

Count the gap only when the earlier reading is strictly above upper_limit_c (8.0 is not heat, 8.1 is).

Code in heat_exposure.py:

```
def heat_increment_s(last_good, record, upper_limit_c):
    """Seconds of heat to add when record arrives. Always an int >= 0."""     if last_good is None:
        return 0     gap_s = record["timestamp_s"] - last_good["timestamp_s"]     if
last_good["temp_c"] > upper_limit_c:
        return gap_s     return 0
```

Result: passes

```
$ python -m unittest test_heat_exposure
test_earlier_reading_above_limit_counts_the_whole_gap ... ok
test_exactly_at_limit_is_not_heat ... ok test_first_reading_adds_nothing ... ok
Ran 3 tests in 0.000s
OK
```

YELLOW PHASE (REFACTOR)

No code change. Checked that strict ">" matches assumption A1 ("above" means strictly greater than) and that the same rule also gives 0 for a reading that is too cold. All 3 tests pass.

Result: still passes

```
$ python -m unittest test_heat_exposure
test_earlier_reading_above_limit_counts_the_whole_gap ... ok
test_exactly_at_limit_is_not_heat ... ok test_first_reading_adds_nothing ... ok
Ran 3 tests in 0.000s
OK
```

Test Case Number: TC-04 | Name: Failed read adds nothing (test_failed_read_adds_nothing) Expected

output: 0 s (the new read failed: ok = false, temp_c = null)

RED PHASE

A failed read still has a timestamp, so the function counted 300 s of heat for a reading with no temperature.

```
def test_failed_read_adds_nothing(self):    self.assertEqual(heat_increment_s(reading(0, 9.0), failed(300), LIMIT), 0)
```

Result: fails

```
$ python -m unittest test_heat_exposure
FAIL: test_failed_read_adds_nothing
AssertionError: 300 != 0
Ran 4 tests in 0.001s
FAILED (failures=1)
```

GREEN PHASE

Return 0 when the new record is not ok.

Code in heat_exposure.py:

```
def heat_increment_s(last_good, record, upper_limit_c):
    """Seconds of heat to add when record arrives. Always an int >= 0."""     if last_good is None:
        return 0     if not record.get("ok"):
        return 0     gap_s = record["timestamp_s"] - last_good["timestamp_s"]     if
last_good["temp_c"] > upper_limit_c:
        return gap_s     return 0
```

Result: passes

```
$ python -m unittest test_heat_exposure
test_earlier_reading_above_limit_counts_the_whole_gap ... ok
test_exactly_at_limit_is_not_heat ... ok test_failed_read_adds_nothing ... ok
test_first_reading_adds_nothing ... ok
Ran 4 tests in 0.000s
OK
```

YELLOW PHASE (REFACTOR)

Moved the "did the read work?" check into its own function, is_good_reading(): ok must be True and temp_c must be a number. Same behaviour, easier to reuse for the earlier reading. All 4 tests pass.

Code after refactoring:

```
def is_good_reading(record):
    """True when the read worked: ok is True and temp_c is a number."""     if record is None or
record.get("ok") is not True:
        return False     temp = record.get("temp_c")     return not isinstance(temp, bool) and isinstance(temp,
(int, float))

def heat_increment_s(last_good, record, upper_limit_c):
    """Seconds of heat to add when record arrives. Always an int >= 0."""     if last_good is None:
        return 0     if not is_good_reading(record):
        return 0
    gap_s = record["timestamp_s"] - last_good["timestamp_s"]     if last_good["temp_c"] >
upper_limit_c:
        return gap_s     return 0
```

Result: still passes

```
$ python -m unittest test_heat_exposure
test_earlier_reading_above_limit_counts_the_whole_gap ... ok
test_exactly_at_limit_is_not_heat ... ok test_failed_read_adds_nothing ... ok
test_first_reading_adds_nothing ... ok
Ran 4 tests in 0.000s
OK
```

Test Case Number: TC-05 | Name: Earlier reading must be a good reading (test_last_good_must_be_good) Expected

output: 0 s, no error (the earlier read failed)

RED PHASE

When the earlier reading had failed, its temp_c is None and comparing None > 8.0 crashed with a TypeError.

```
def test_last_good_must_be_good(self):    self.assertEqual(heat_increment_s(failed(0), reading(300, 9.0), LIMIT), 0)
```

Result: fails (TypeError)

```
$ python -m unittest test_heat_exposure
ERROR: test_last_good_must_be_good
File "heat_exposure.py", line 20, in heat_increment_s    if last_good["temp_c"] >
upper_limit_c:
TypeError: '>' not supported between instances of 'NoneType' and 'float'
Ran 5 tests in 0.001s
FAILED (errors=1)
```

GREEN PHASE

Return 0 when the earlier reading is not a good reading.

Code in heat_exposure.py:

```
def heat_increment_s(last_good, record, upper_limit_c):
   """Seconds of heat to add when record arrives. Always an int >= 0."""    if not is_good_reading(last_good)
or not is_good_reading(record):
      return 0    gap_s = record["timestamp_s"] - last_good["timestamp_s"]    if
last_good["temp_c"] > upper_limit_c:
      return gap_s    return 0
```

Result: passes

```
$ python -m unittest test_heat_exposure
test_earlier_reading_above_limit_counts_the_whole_gap ... ok
test_exactly_at_limit_is_not_heat ... ok
test_failed_read_adds_nothing ... ok
test_first_reading_adds_nothing ... ok
test_last_good_must_be_good ... ok
Ran 5 tests in 0.000s OK
```

Test Case Number: TC-06 | Name: Time going backwards adds nothing (test_time_going_backwards_adds_nothing)

Expected output: 0 s (clock went from 5000 s back to 10 s, as after a power cut)

RED PHASE

After a restart the clock can go back. The gap was negative (-4990), which would lower the total (breaks AC2).

```
def test_time_going_backwards_adds_nothing(self):    self.assertEqual(heat_increment_s(reading(5000, 9.0), reading(10, 5.0), LIMIT), 0)
```

Result: fails

```
$ python -m unittest test_heat_exposure
FAIL: test_time_going_backwards_adds_nothing
AssertionError: -4990 != 0
Ran 6 tests in 0.001s
FAILED (failures=1)
```

GREEN PHASE

Return 0 when the gap is 0 or less.

Code in heat_exposure.py:

```
def heat_increment_s(last_good, record, upper_limit_c):    if not is_good_reading(last_good) or not
is_good_reading(record):        return 0    gap_s = int(record["timestamp_s"]) -
int(last_good["timestamp_s"])    if gap_s <= 0:        return 0    if last_good["temp_c"] > upper_limit_c:
      return gap_s    return 0
```

Result: passes

```
$ python -m unittest test_heat_exposure test_time_going_backwards_adds_nothing ... ok
Ran 6 tests in 0.000s
OK
```

YELLOW PHASE (REFACTOR)

Final clean-up: whole-second timestamps with int(), complete docstrings listing all zero conditions, and Rule R1 documentation matching GitHub standard (heat_exposure/heat_exposure.py). All 17 tests in the full suite pass cleanly.

Result: still passes (all 17 tests OK)

```
$ python -m unittest test_heat_exposure .................
Ran 17 tests in 0.001s OK
```